



RELATÓRIO TÉCNICO

BACKEND — ARQUITETURA & SEGURANÇA

Rotas, auth, rate limiting, filas, testes e infraestrutura

VERSÃO	DATA	MÓDULOS	LOC
v25	15/07/2026	17	25.9k



Registro de Rotas

MÓDULO	LOC	AUTENTICAÇÃO	DESCRIÇÃO
studio	809	Público	Geração e edição de criativos
openrouter	503	Público	Proxy para LLMs via OpenRouter
goals	445	Público	Metas de campanha (KPIs)
billing	320	Público	Planos e pagamento
fury	249	Público	Fury Engine endpoints
observability	223	Protegido	Logs e métricas operacionais
brand-kit	219	Protegido	Identidade visual do tenant
budget	63	Público	Orçamento de campanha
superadmin	63	Público	Painel administrativo
campaigns	60	Protegido	CRUD de campanhas
meta	42	Público	Integração Meta Ads API
metrics	41	Protegido	Métricas de anúncios
automation	24	Público	Regras de automação
forms	23	Protegido	Formulários de lead gen
auth	21	Público	Registro, login, JWT
instagram	9	Protegido	Proxy Instagram API

MÓDULO	LOC	AUTENTICAÇÃO	DESCRIÇÃO
dashboard	8	Protegido	Dados agregados

AUTH-01 Auth · Design

Ativo

Tokens rotativos com hash SHA-256 armazenado no Redis

`access token` expira em **15 minutos**. `refresh token` expira em **7 dias**. Cada refresh token é hashado (SHA-256) e persistido no Redis com TTL de 7 dias. Na rota `POST /auth/refresh`, o hash é comparado com `crypto.timingSafeEqual` para evitar timing attacks. No logout, a hash é removida do Redis imediatamente.

Segredos: `JWT_SECRET` e `JWT_REFRESH_SECRET` separados, ambos obrigatórios (crash startup se ausentes).

AUTH-02 Auth · Middleware

Ativo

Middleware de autenticação duplo (JWT + mock quickstart)

`authMiddleware`: extrai Bearer token, verifica com `verifyAccessToken`. Em desenvolvimento (`AUTH_BYPASS_DEV=true` + `NODE_ENV≠production`), aceita tokens mock (base64 JSON com `userId`).

`authSSEMiddleware`: variante para EventSource — aceita token por query string ou header.

AUTH-03 Auth · Tenant

Ativo

Tenant middleware — isolamento multi-tenant

`tenantMiddleware`: garante que todo request autenticado tenha `tenantId` no JWT. Se ausente, retorna **403** (não 401 — o usuário está autenticado, mas sem contexto de tenant). Isso impede vazamento de dados entre tenants.

AUTH-04 Auth · Rotas

Ativo

Rotas públicas vs protegidas

Públicas (10): `/auth`, `/health`, `/meta`, `/studio`, `/fury`, `/goals`, `/billing`, `/budget`, `/automation`, `/openrouter`.

Protegidas (7): `/campaigns`, `/dashboard`, `/metrics`, `/instagram`, `/forms`, `/brand-kit`, `/observability`.

`/admin` usa proteção por role (superadmin) — rota separada.

SEC-01 Rate Limit · Redis

Ativo

Sliding window atômico no Redis (Lua script)

Limite padrão: **100 requisições por segundo** por IP. A janela é de 1 segundo (configurável via `RATE_LIMIT_WINDOW_MS`). O Redis executa um script Lua atômico: `ZREMRANGEBYSCORE` → conta entradas na janela → se $\geq$ limite, rejeita; senão adiciona entrada com timestamp. Resposta 429 inclui headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `Retry-After`.

IP real: usa `x-forwarded-for` (proxy reverso) → `req.ip` → `socket.remoteAddress`.

Desabilitável: `RATE_LIMIT_ENABLED=false` ou `NODE_ENV=test`. Health checks (`/health`) nunca consomem cota.

SEC-02 CORS · Express

Ativo

CORS dinâmico com credentials

CORS configurado com callback que aceita qualquer origin (`callback(null, true)`) + `credentials: true`. Adequado para SaaS multi-tenant onde domínios customizados são frequentes.

Atenção: `allow-origin: *` com credentials expõe cookies de sessão. Como a API usa Authorization header (Bearer), o risco é mitigado. Em produção com cookies, restringir origem.

SEC-03 Error Handler · Segurança

Ativo

Sem vazamento de stack traces

O `errorHandler` retorna sempre `{ success: false, error: { code, message } }`. Para erros inesperados, retorna `INTERNAL_SERVER_ERROR` com mensagem genérica (*"An unexpected error occurred"*). Stack traces não são expostos. Erros de validação Zod retornam array de erros por campo, mas sem dados sensíveis.

SEC-04 Auth · Refresh

Ativo

Refresh token armazenado como hash (não plaintext)

O refresh token **nunca é armazenado em texto plano**. É hashado com SHA-256 e o hash é salvo no Redis com chave `refresh:{userId}`. A comparação usa `timingSafeEqual` — resistente a timing attacks. No logout, a hash é removida (`DEL`). Se um atacante obtiver o hash, não pode derivar o token original.

bcrypt com salt rounds

Senhas armazenadas com `bcrypt.hash(password, 12)`. Comparação com `bcrypt.compare`. Email verificado via OTP de 6 dígitos antes de liberar acesso completo. Reset de senha via token único enviado por email.

INFRA-01 Queue · Design

Ativo

4 filas BullMQ independentes com singleton Redis

Cada fila usa uma **conexão Redis dedicada** (`duplicate()`) obtida via singleton lazy-loading. A conexão é reutilizada — se já estiver criando, aguarda a mesma Promise.

Filas ativas:

- `compliance-check` — validação de conformidade de anúncios
- `studio-generation` — geração de criativos (GPU-bound, assíncrono)
- `budget-optimizer` — otimização de orçamento
- `sync-jobs` — sincronização periódica Meta ↔ banco local

Engines (schedulers):

- `rule-engine` — dispara regras de automação
- `fury-engine` — engine principal FURY

INFRA-02 Server · Config

Ativo

Timeouts HTTP ajustados para requisições longas

`server.timeout = 60s` , `keepAliveTimeout = 65s` , `headersTimeout = 66s` . Evita que o Traefik retorne **502 Bad Gateway** antes do Node.js concluir geração de imagem (que pode levar 20-40s).

INFRA-03 Docker · Deploy

Ativo

Container com healthcheck, restart e network isolada

`docker-compose.api.yml` expõe porta 3001 → 3000, com healthcheck HTTP a cada 30s.

`network_mode: infra_fury-network` (rede externa compartilhada com Redis e PostgreSQL).

`extra_hosts` mapeia IPs fixos para `fury-postgres` e `fury-redis` . `restart: unless-stopped` .

INFRA-04 Seed · Startup

Ativo

Seed automático na inicialização

`seedStartup()` roda antes do servidor aceitar requests — garante que superadmin + tenants demo existam. Ambiente `test` pula seed e workers.

F**Nota Metodológica**

Dados extraídos do código-fonte em `click-hero/fury-app-v2/apps/api`. Métricas de LOC incluem apenas TypeScript (*.ts). Contagem de rotas, middlewares e workers baseada em arquivos fonte, não em registro dinâmico. Testes contabilizados: 30 suites, 5.379 linhas — coverage estimado em ~20%. Relatório gerado em 15/07/2026.

Alta prioridade — baixo esforço

REC-01 Roteamento · Acesso

Recomendado

Aplicar auth nas rotas públicas de billing e budget

`/billing` e `/budget` operam dados financeiros sem middleware de auth. Basta adicioná-los ao array `AUTH_TENANT_SUB` em `routes/index.ts` — uma linha por rota.

Impacto: evita exposição de dados de faturamento e orçamento a requests não autenticados.

Esforço: ~2 linhas.

REC-02 CORS · Segurança

Recomendado

Restringir CORS a origens conhecidas em produção

Hoje `callback(null, true)` aceita qualquer origin. Em produção, validar contra lista de domínios permitidos (tenant custom domains + domínio principal). Fallback para `null` só em dev.

Impacto: reduz superfície de CSRF. **Esforço:** ~15 linhas com array de origins + env var.

REC-03 Testes · Qualidade

Recomendado

Dobrar cobertura de testes — mirar 40%

Atualmente ~20% (5.379 linhas de teste para 25.932 de código). Priorizar testes unitários nos services: `auth.service`, `campaigns.service`, `meta.service`, `studio-image.service`. Cada service com 2-3 cenários de sucesso + 1 de erro.

Impacto: reduz regressões em deploys. **Esforço:** ~30-40 testes novos, 2-3 dias de trabalho.

Média prioridade — médio esforço

Rate limiting por rota, não global

O sliding window atual é global (100 req/s por IP em todas as rotas). Separar limites: `/auth/register` (5/min), `/auth/login` (10/min), `/studio` (20/min, é GPU-bound), rotas CRUD (100/s mantido). A estrutura de middleware já suporta — adicionar chave por rota no Redis.

Impacto: protege endpoints sensíveis sem estrangular uso normal. **Esforço:** refatorar `rateLimitMiddleware` para aceitar config por path — ~40 linhas.

Adicionar request ID e tracing básico

O logger atual (`console.log`) não inclui `requestId` nem `tenantId`. Adicionar middleware que gera UUID por request e injeta no `req` + header `X-Request-Id`. Permite rastrear erros ponta a ponta.

Impacto: debug de produção muito mais rápido. **Esforço:** ~20 linhas de middleware + adoção no error handler.

Remover mock token de produção

`AUTH_BYPASS_DEV` é seguro (só funciona com `NODE_ENV≠production`), mas o código de mock token fica no bundle de produção. Extrair para módulo separado que só é importado em dev — evita superfície de ataque.

Impacto: hardening — nenhuma vulnerabilidade ativa, mas reduz superfície. **Esforço:** extrair função mock para arquivo condicional.

Timeout progressivo nas chamadas OpenRouter

`studio.routes.ts` (809 linhas) é o arquivo mais extenso. Geração de imagem pode levar 20-40s. Adicionar timeout configurável por provider + feedback parcial (SSE) enquanto imagem é gerada — evita que o cliente veja "loading infinito".

Impacto: UX do estúdio mais responsiva. **Esforço:** SSE já existe (`authSSEMiddleware`) — adaptar.

Baixa prioridade — quando houver tempo

REC-08 Docker · Infra

Opcional

Migrar extra_hosts para DNS interno

IPs fixos (172.26.0.3 , 172.26.0.2) quebram se containers reiniciarem em ordem diferente. Usar nomes de serviço DNS do Docker Compose (rede compartilhada resolve por container name).

Impacto: elimina falha de deploy por IP fixo. **Esforço:** baixo — só remover extra_hosts e confiar no DNS interno.

REC-09 Rate Limit · Escala

Futuro

Substituir sliding window por token bucket se escala exigir

Sliding window com ZSET escala bem até ~10K req/s por Redis. Acima disso ou com custo de Redis elevado, migrar para token bucket (1 chave string + INCR + EXPIRE). Não necessário agora.

Gatilho: latência de Redis >5ms no rate limit ou custo de infra subindo. **Esforço:** ~30 linhas, swap do script Lua.

REC-10 Queue · Resiliência

Futuro

Dead letter queue para jobs que falham ≥3x

Jobs que falham repetidamente são descartados pelo BullMQ. Adicionar DLQ para inspeção manual — evita perda silenciosa de jobs.

Impacto: visibilidade de falhas. **Esforço:** configurar removeOnComplete e handler de falha — ~20 linhas.

F

Critérios de Priorização

Alta / Baixo esforço: correções que previnem incidentes, exigem ≤1 dia.

Média / Médio esforço: melhorias estruturais, 2-5 dias cada.

Baixa / Quando houver tempo: otimizações e hardening sem risco imediato.

Recomendações baseadas em auditoria estática do código em 15/07/2026. Nenhum item é bloqueante — todos são evolutivos.